

Name : **SALAMI SAMUEL OLUWATOMIWA**

Course: CPE 301 Introduction to low level languages

Matric No: CSC/2021/176

Title: Assignment Project Lab 1 and Lab 2

Lecturer: Dr Ayodeji

Department: Computer science with mathematics

Lab 1 solutions.

Instruction: Display “ Great lfe, Oba Awon University” with the background color = light blue and foreground color = green

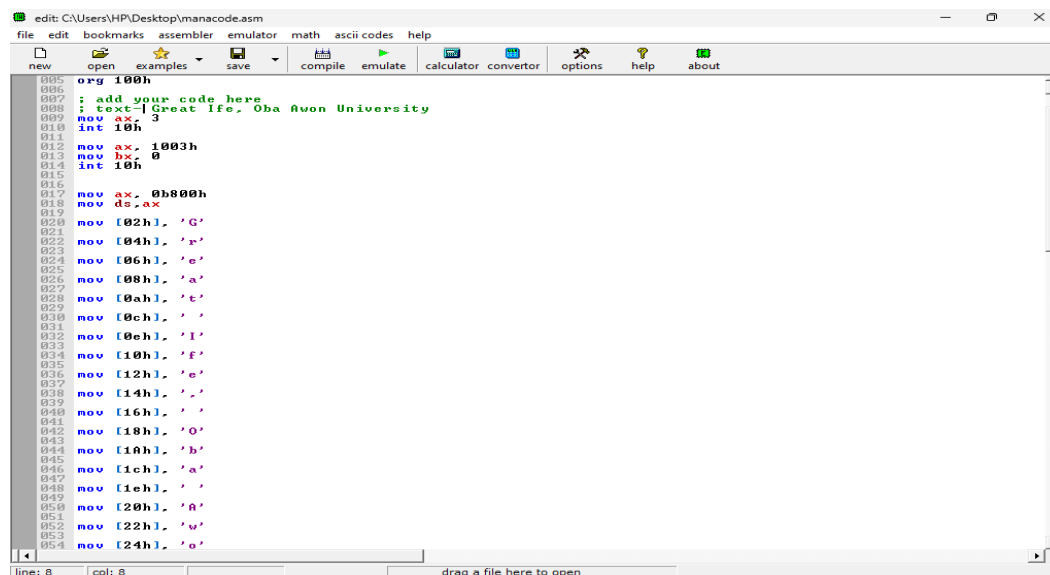
Procedure:

1. Write out the code setting to video text mode

mov ax, 3

int 10h

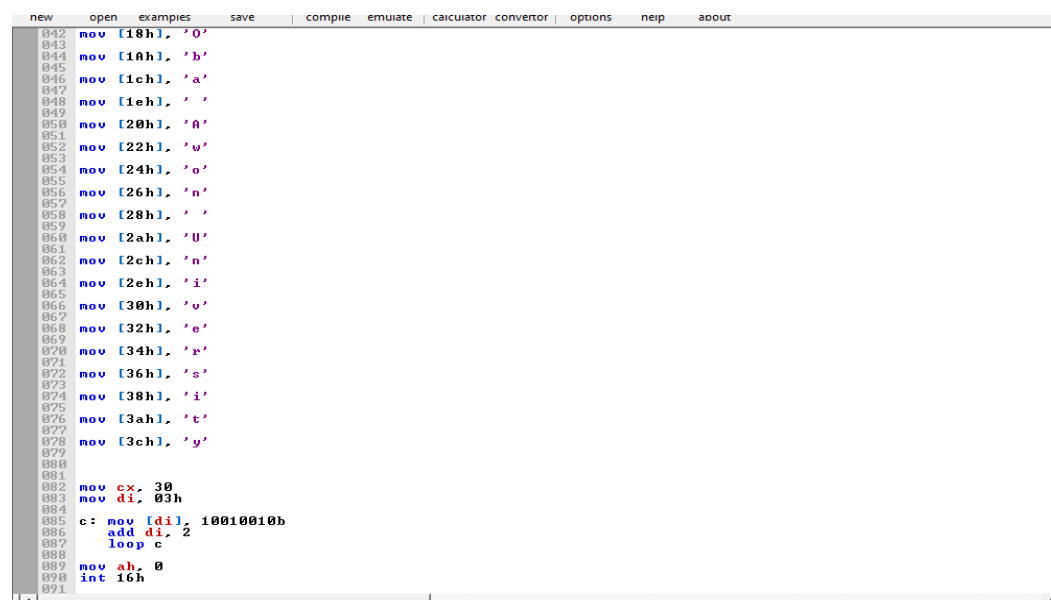
set it to video text mode.



```

edit: C:\Users\HP\Desktop\manacode.asm
file  edit  bookmarks  assembler  emulator  math  ascii codes  help
new  open  examples  save  compile  emulate  calculator  convertor  options  help  about
0005 org 100h
0006 ; add your code here
0007 ; text-| Great lfe, Oba Awon University
0008 mov ax, 3
0009 int 10h
0010
0011 mov ax, 1003h
0012 mov bx, 0
0013 int 10h
0014
0015
0016
0017 mov ax, 0b800h
0018 mov ds, ax
0019
0020 mov [02h], 'G'
0021 mov [04h], 'r'
0022
0023 mov [06h], 'e'
0024
0025 mov [08h], 'a'
0026
0027 mov [0ah], 't'
0028
0029 mov [0ch], ' '
0030
0031 mov [0eh], 'I'
0032
0033 mov [10h], 'f'
0034
0035 mov [12h], 'e'
0036
0037 mov [14h], ' '
0038
0039 mov [16h], ' '
0040
0041 mov [18h], 'O'
0042
0043 mov [1ah], 'b'
0044
0045 mov [1ch], 'a'
0046
0047 mov [1eh], ' '
0048
0049 mov [20h], 'A'
0050
0051 mov [22h], 'u'
0052
0053 mov [24h], 'o'
0054
0055
0056 mov [26h], 'n'
0057
0058 mov [28h], ' '
0059
0060 mov [2ah], 'U'
0061
0062 mov [2ch], 'n'
0063
0064 mov [2eh], 'i'
0065
0066 mov [30h], 'v'
0067
0068 mov [32h], 'e'
0069
0070 mov [34h], 'r'
0071
0072 mov [36h], 's'
0073
0074 mov [38h], 'i'
0075
0076 mov [3ah], 't'
0077
0078 mov [3ch], 'y'
0079
0080
0081
0082 mov cx, 30
0083 mov di, 03h
0084
0085 c: mov [di], 10010010h
0086 add di, 2
0087 loop c
0088
0089 mov ah, 0
0090 int 16h
0091

```



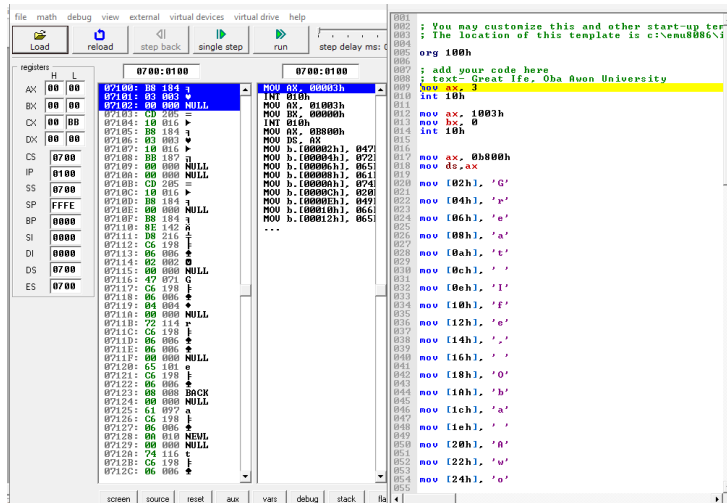
```

new  open  examples  save  compile  emulate  calculator  convertor  options  help  about
0042 mov [18h], 'O'
0043
0044 mov [1ah], 'b'
0045
0046 mov [1ch], 'a'
0047
0048 mov [1eh], ' '
0049
0050 mov [20h], 'A'
0051
0052 mov [22h], 'u'
0053
0054 mov [24h], 'o'
0055
0056 mov [26h], 'n'
0057
0058 mov [28h], ' '
0059
0060 mov [2ah], 'U'
0061
0062 mov [2ch], 'n'
0063
0064 mov [2eh], 'i'
0065
0066 mov [30h], 'v'
0067
0068 mov [32h], 'e'
0069
0070 mov [34h], 'r'
0071
0072 mov [36h], 's'
0073
0074 mov [38h], 'i'
0075
0076 mov [3ah], 't'
0077
0078 mov [3ch], 'y'
0079
0080
0081
0082 mov cx, 30
0083 mov di, 03h
0084
0085 c: mov [di], 10010010h
0086 add di, 2
0087 loop c
0088
0089 mov ah, 0
0090 int 16h
0091

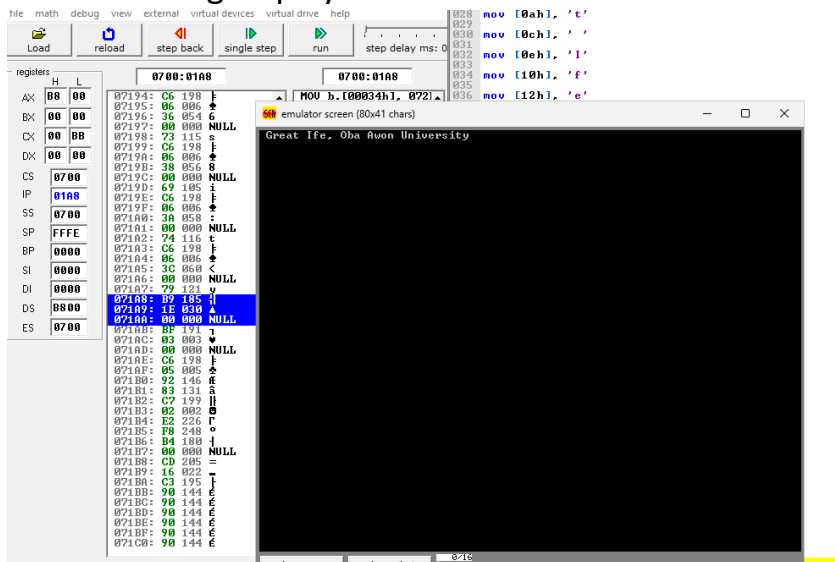
```

The ds register holds the data segment for the ascii equivalent of each letter, e.g. the ascii character 'G' is stored in [02h] data segment etc.

2. Loop through each letter and print out on the screen.



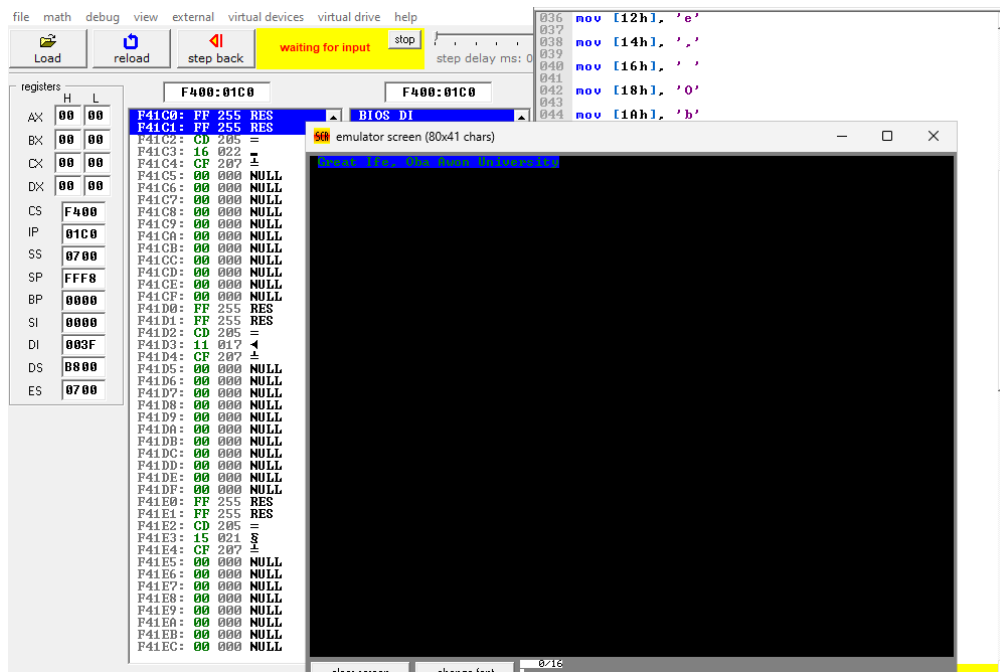
The above image is showing how the text is being looped and the ascii code or letter being displayed on the screen one letter at a time.



3. The color for this was background color light blue & foreground color = green, which is represented as 10010010

The image shown above shows the process of text being covered by the color.

The final result will look like this:



Conclusion: In order to be able to add color and background color to the text, I first set video text mode then I looped through each text and display the ascii characters on screen then loop again this time adding the colors and background color to it to get my desired output.

Lab 2 solutions

1. Write an assembly program to sum five numeric 10, -200, 36, 00, -255 The program for the question above is shown below:

The program begins by clearing the AX and BX registers, then iterates through the array of numbers, accumulating their sum in AX. The loop counter CX is initialized to 5, matching the array's length. Meanwhile, the SI register is assigned the starting address of the array, enabling the code to advance through each element on every iteration.

```

05  org 100h
06
07  ; Question 1: Write an assembly program to sum five numerics [10, -200, 36, 00, -255]
08
09  number DW 10, -200, 36, 00, -255
10
11  ; ax will store the sum of the values
12  mov ax, 0
13
14  ; cx is the number of elements/loop
15  mov cx, 5
16  mov si, offset number
17
18  sum_loop:
19      add ax, [si]
20      add si, 2
21      loop sum_loop
22

```

Figure: A snippet of the code that sums the five numeric values and stores the result in the AX register.

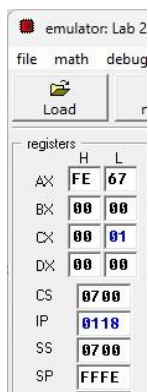


Figure: Screenshot of the register after execution of the program (The AX register stores FE67 which is the result of the program)

2. Multiply the first three numbers

The code begins by setting AX to 1 and SI to the start of the numerics array. It then iterates three times (CX = 3), loading each element into BX and multiplying AX by BX on each pass, storing the updated product in AX.

```

24 ; Question 2: Write an assembly program to multiply the first three numbers in number
25
26 mov ax, 1
27
28 mov cx, 3
29 mov si, offset number
30
31 mul_loop:
32     mov bx, [si]
33     mul bx
34     add si, 2
35     loop mul_loop
36

```

Figure: The code showing multiplication of the first three array values by repeatedly loading each into BX and multiplying AX by BX.

registers		H	L
AX	E6	C0	
BX	00	24	
CX	00	01	
DX	00	22	
CS	0700		
IP	012A		
SS	0700		
SP	FFFE		

Snapshots of the register post-execution

3. Multiply the first and the third number using the SHL and ADD rather than the MUL

To accomplish this, we first load the first item's value into AX. Since the third element is 36, which breaks down to $32 + 4$ (both powers of two), we multiply by shifting left 5 bits ($\times 32$), then

2 bits ($\times 4$), and combine the results. The code is shown below

```

38 ; Question 3: multiply the first and the third number using the SHL and ADD rather than the MUL
39
40 mov si, offset number
41 mov ax, [si]
42 mov bx, [si]
43 shl bx, 2
44 shl ax, 5
45 add ax, bx
46

```

Figure: The code demonstrating multiplication of the first and third numbers in the array by shifting and adding (rather than using `mul`).

registers		
	H	L
AX	01	68
BX	00	28
CX	00	00
DX	00	22
CS	0700	
IP	0143	
SS	0700	
SP	FFFE	

Snapshots of the register post-execution

4. Divide the second by the third

```

48 ; Question 4: divide the second by the third
49
50 mov si, offset number
51 mov bx, 0
52 mov ax, 0
53 mov ax, [si + 2]
54 mov bx, [si + 4]
55 div bx
56

```

Figure: Code showing how to divide the second array element by the third by loading each into registers (AX for the dividend, BX for the divisor) and then performing the div bx instruction.

registers		
	H	L
AX	FF	38
BX	00	24
CX	00	00
DX	00	22
CS	0700	
IP	0152	
SS	0700	
SP	FFFE	

Snapshots of the register post-execution

5. Divide the third by 8 using the SHR rather than the DIV.

```

57 ; Question 5: divide the third by 8 using the SHR rather than the DIV
58
59 mov ax, [number + 4]
60 shr ax, 3
61
62 ret

```

Figure: Code showing how to divide the second array element by the third by loading each into registers (AX for the dividend, BX for the divisor) and then performing the `div bx` instruction.

registers		H	L
AX	00	04	
BX	00	24	
CX	00	00	
DX	00	00	
CS	07 00		
IP	01 5D		
SS	07 00		
SP	FF FE		

Snapshots of the register post-execution